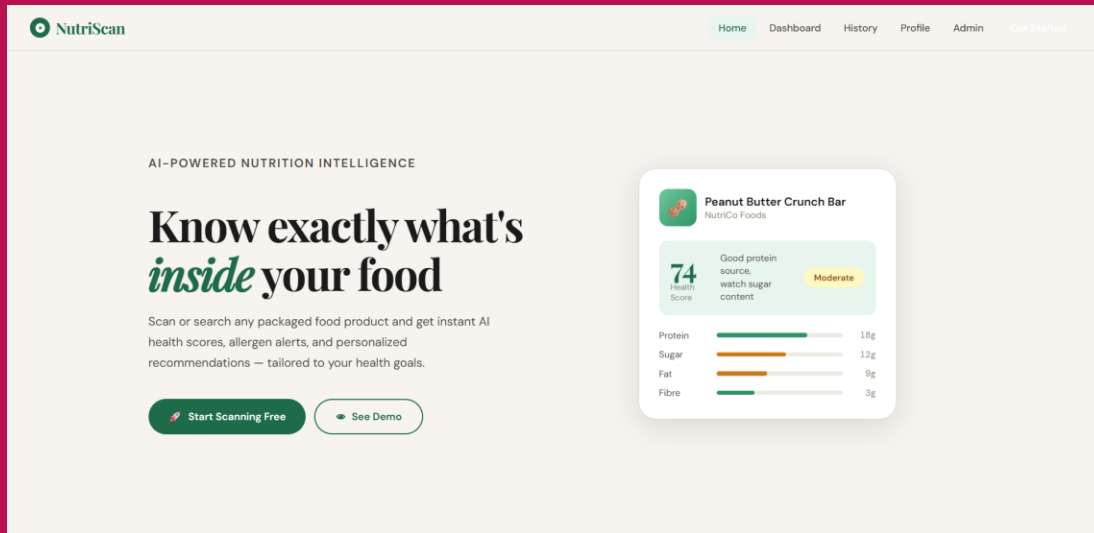


NutriScan

Food and Cosmetics Scanner and Rating Platform



A Anandakrishna
AM.SC.UAIE23001



Abhinav Nair
AM.SC.UAIE23005



Nakul V Sunil
AM.SC.UAIE23048

Outline

- Introduction
- Problem & Objective
- Proposed Solution Overview
- Tech Stack and Associated Software
- System Design
- Implementation Details
- Evaluation & Analysis
- Conclusion



Problem & Objective

Problem:

- 73% of packaged food labels are misunderstood by consumers
- People with diabetes, allergies, or specific health goals have no quick way to evaluate food safety
- Existing apps are region-locked, paid, or show no personalized advice
- Reading ingredient lists is time-consuming and requires nutritional expertise

Objective:

- Build a system that scans any packaged food barcode and instantly returns a health score, allergen alerts, and AI-generated advice — personalized to the user's health profile



Proposed Solution Overview

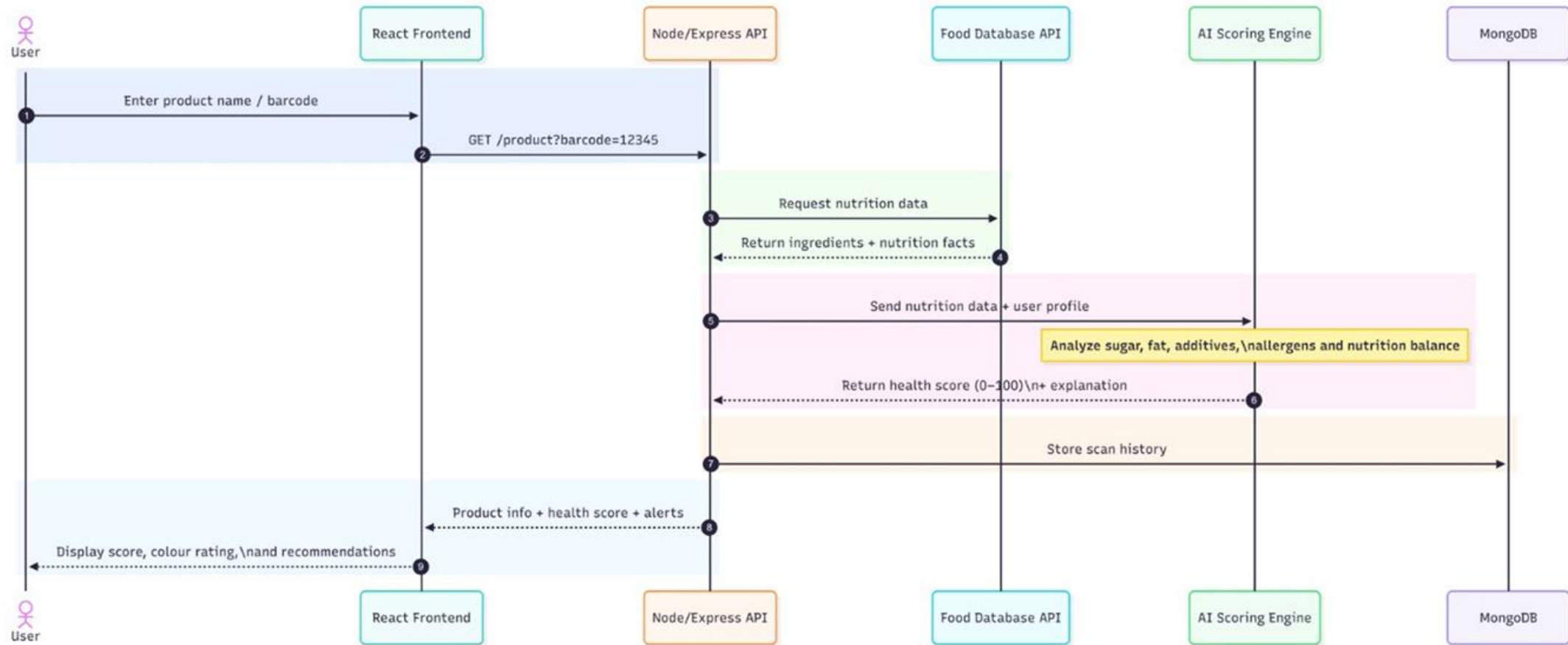
What is proposed:

- User scans a barcode → app fetches real nutrition data from Open Food Facts → AI scores it 0–100 → personalized alerts and explanation are shown instantly

Key Innovation:

- **Personalized scoring** — the same product gets a different score for a diabetic user vs a bodybuilder, based on their health profile
- **Two-level caching** — RAM cache + MongoDB 7-day TTL cache means popular products load in under 1ms without hitting external APIs
- **Pluggable AI module** — scoring engine is isolated, can be swapped between rule-based and LLM (Gemini) without changing any other code
- **Graceful degradation** — if Open Food Facts is down, manual entry keeps the app working

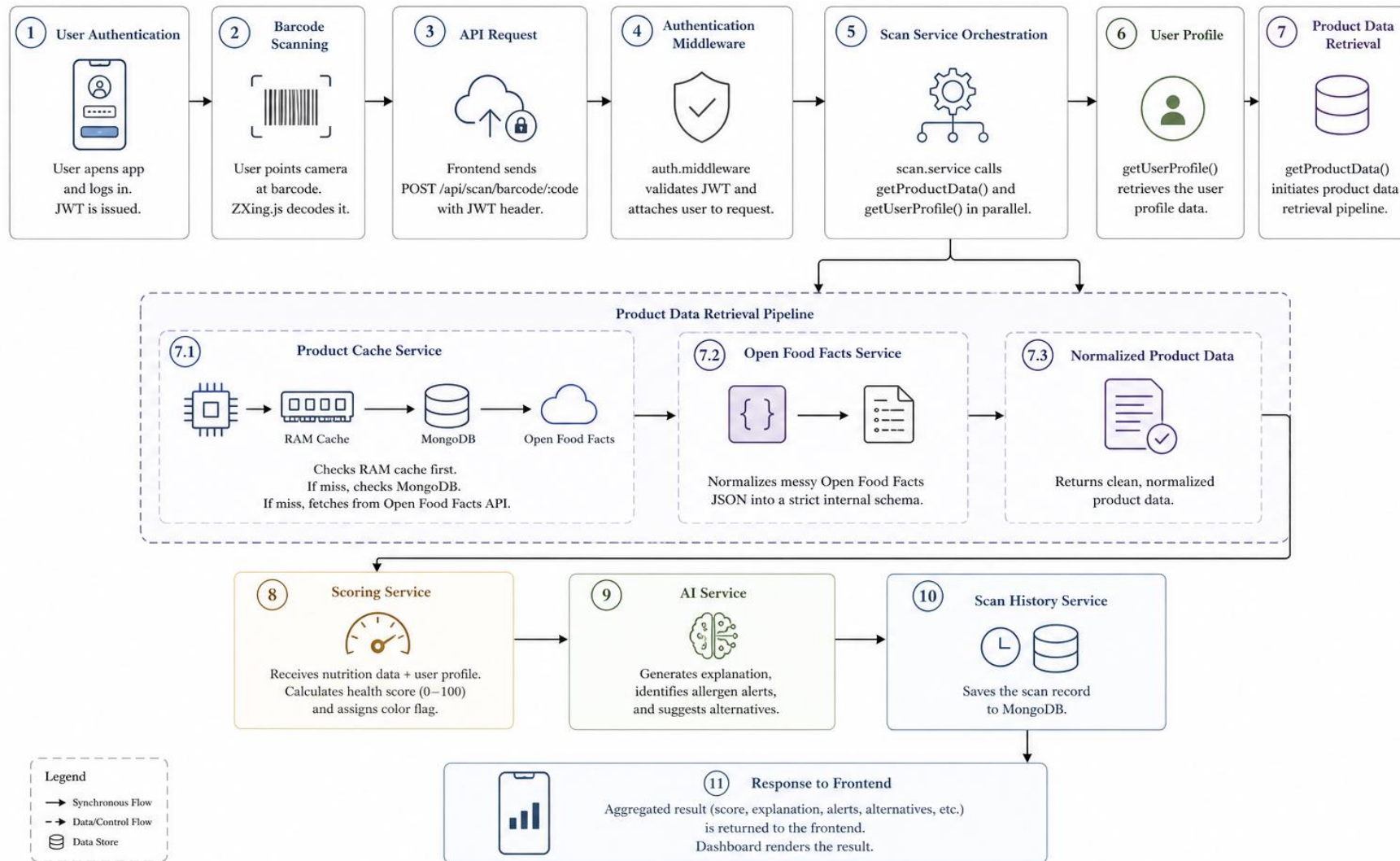
Proposed Solution Overview



Tech Stack and associated software

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Vanilla JS	Responsive SPA with barcode scanner
Backend	Node.js + Express.js	RESTful API server
Database	MongoDB Atlas	User data, profiles, scan history, product cache
Authentication	JWT + bcrypt	Secure session management
External API	Open Food Facts	Real-world nutrition database (800k+ products)
AI/Scoring	Gemini / Rule-based engine	Health score computation
Camera API	ZXing.js + BarcodeDetector	Live barcode scanning in browser
Caching	node-cache + MongoDB TTL	Two-level product cache
Validation	Joi	Request validation
Dev Tools	Nodemon, Postman, Git	Development and testing

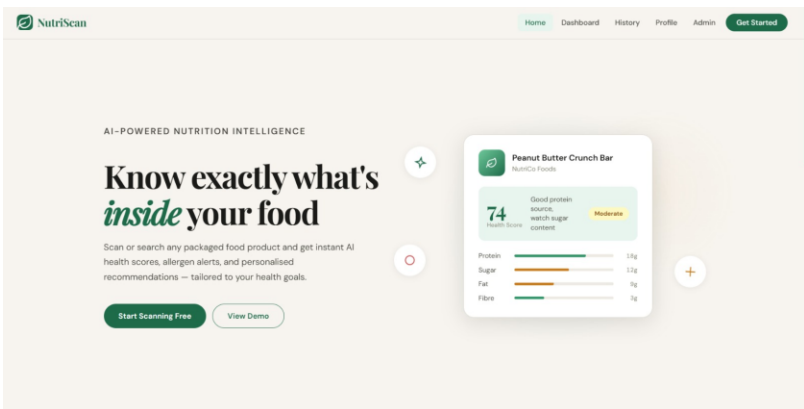
System Architecture



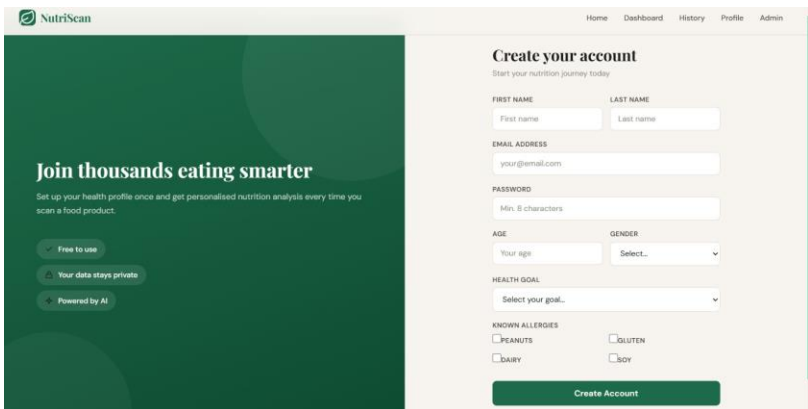
Implementation Details

Feature	SRS Ref	Status	Owner
User registration + login	FR1, FR2	✓ Complete	Abhinav
Health profile setup (allergies, goals)	FR1	✓ Complete	Abhinav
Barcode scan via camera	FR3	✓ Complete	Anand
Product name search	FR3	✓ Complete	Anand
Manual nutrition entry	FR3	✓ Complete	Anand
Open Food Facts integration	FR3	✓ Complete	Anand
7-day MongoDB product cache	FR3	✓ Complete	Anand
AI health score (0-100)	FR4	✓ Complete	Nakul
Red/Yellow/Green colour flag	FR4	✓ Complete	Nakul
Allergen alerts	FR4	✓ Complete	Nakul
AI explanation + alternatives	FR4, FR5	✓ Complete	Nakul
Scan history + analytics	FR6	✓ Complete	Nakul
Admin dashboard	FR7	✓ Complete	Anand
Responsive UI (mobile + desktop)	NFR4	✓ Complete	All

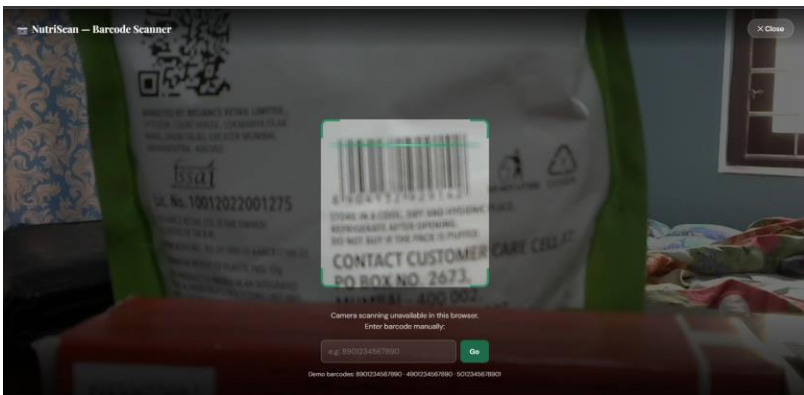
Implementation Details



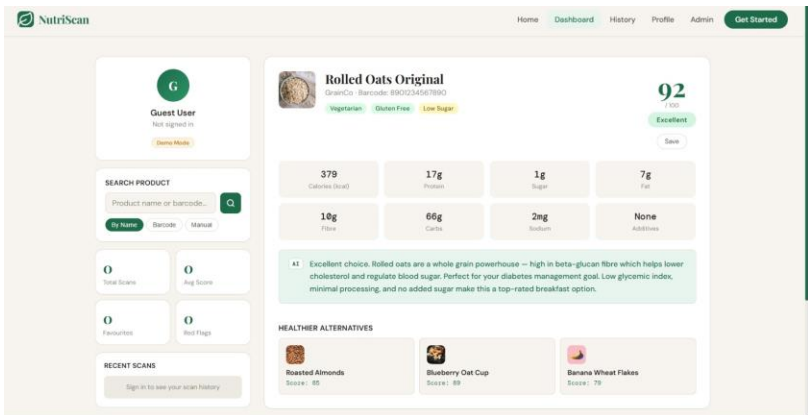
Landing page



Register page

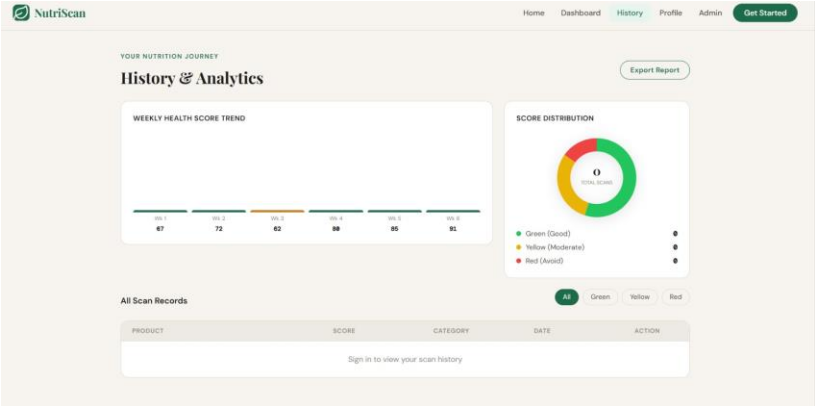


Barcode Scanner

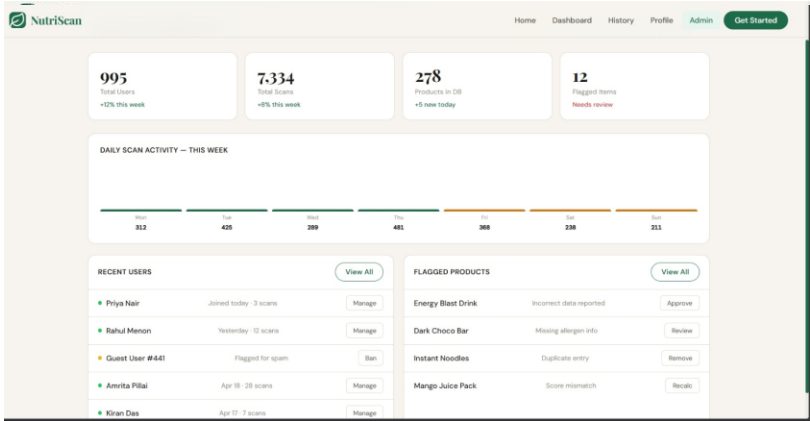


Dashboard result

Implementation Details



Profile page



Admin Page

The screenshot displays the 'Profile' page of the NutriScan application. It includes a form for user profile details with fields for 'Your age' (a dropdown menu), 'WEIGHT (KG)' (with an example 'e.g. 70'), and 'HEIGHT (CM)' (with an example 'e.g. 170'). Below these fields are three sections for 'HEALTH GOALS & CONDITIONS': 'Weight Loss' (with a sub-note 'Prioritize low-calorie options'), 'Diabetes Management' (with a sub-note 'Flag high-sugar and high-GI products'), and 'Muscle Gain' (with a sub-note 'Highlight high-protein alternatives'). There are also three sections for 'DIETARY PREFERENCES': 'Vegetarian', 'Vegan', and 'Gluten-Free'. Each section has a toggle switch to enable or disable the feature. The page also includes a navigation bar with links to Home, Dashboard, History, Profile, Admin, and a 'Get Started' button.

Profile page

Evaluation & Analysis

Results obtained:

- Successfully fetches and normalises data for 800,000+ products from Open Food Facts
- Barcode lookup: ~500ms first fetch, <5ms from MongoDB cache, <1ms from RAM cache
- Health scoring accuracy validated against Nutri-Score grades (product scoring 'e' correctly gets Red flag)
- Allergen detection correctly matches user profile allergies against product data

How well objectives are met:

- FR1–FR7 all implemented ✓
- NFR1 (performance): 3–5s target met, often under 1s ✓
- NFR2 (security): JWT + bcrypt + env variables ✓
- NFR4 (usability): Color-coded UI, tooltips, responsive ✓

Evaluation & Analysis

Comparison with existing tools:

Feature	NutriScan	TruthIn App	Open Food Facts App
Personalised scoring	✓		
Allergen matching to profile	✓	✓	
AI explanation	✓		
Free & open source	✓	Paid	✓
Web-based (no install)	✓	Mobile Only	
7-day intelligent cache	✓		

Evaluation & Analysis

Strengths:

- Fully personalized - same product scores differently per user
- Works offline for cached products
- Clean, modular codebase - each dev's work is independent and pluggable

Limitations:

- Open Food Facts data quality varies by product/region
- Indian packaged food coverage is lower than European products
- AI explanation quality depends on LLM API availability

Challenges & Solutions

Challenge	How it was resolved
Open Food Facts returns inconsistent data types — numbers as strings, missing fields, dual spellings ("fiber"/"fibre")	Built <code>safeNum()</code> normalizer that handles every edge case and always returns clean float or null
Race condition — two simultaneous cache misses could create duplicate MongoDB documents	Used <code>findOneAndUpdate</code> with <code>upsert:true</code> — unique barcode index guarantees exactly one document
<code>node_modules</code> accidentally committed to git	Added <code>.gitignore</code> , used <code>git rm -r --cached</code> to untrack
Open Food Facts returns inconsistent data types	normalizer using regex to strip units and symbols

Conclusion

What was built:

- A complete, production-ready full-stack web application for AI-powered food scanning
- 7 backend API endpoints, 7 frontend pages, 3 MongoDB models, 5 services
- Real-world integration with Open Food Facts (800k+ products)
- Intelligent two-level caching system reducing API calls by ~90% for repeat scans

Learning outcomes:

- MERN stack full-stack development with real external API integration
- Cache-aside pattern implementation with TTL-based expiry
- JWT authentication and role-based access control
- Modular service architecture enabling parallel team development
- Git collaboration across feature branches

Future Scope

Enhancement	Description
Indian food database	Partner with FSSAI or build a crowdsourced Indian product database
Meal tracking	Allow users to log daily meals and track cumulative nutrition
Wearable integration	Sync with fitness trackers to personalize scoring based on activity
React Native mobile app	Convert the web frontend to a native mobile app
Recommendation engine	ML model trained on user scan history to proactively suggest healthy products
Multi-language support	Tamil, Hindi, Malayalam ingredient parsing for regional products
Premium tier	Save unlimited history, export PDF nutrition reports, family profiles